

Chapter 4. Cluster Installation and Upgrade

The first domain of the curriculum refers to typical tasks you'd expect of a Kubernetes administrator. Those tasks include understanding the architectural components of a Kubernetes cluster, setting up a cluster from scratch, and maintaining a cluster going forward. At the end of this chapter, you will understand the tools and procedures for installing and maintaining a Kubernetes cluster.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objectives:

- Prepare underlying infrastructure for installing a Kubernetes cluster
 - Understand extension interfaces (CNI, CSI, CRI, etc.)
 - Create and manage Kubernetes clusters using `kubeadm`
 - Manage the lifecycle of Kubernetes clusters
 - Implement and configure a highly available control plane
-

Provisioning the Infrastructure

The infrastructure required by Kubernetes cluster nodes consist of servers, networks, and storage. Provisioning of the infrastructure is done by configuring those resources on premises or in the cloud, optimally in an automated fashion. That's the purpose of infrastructure automation tools like Ansible and Terraform.

Even though “provisioning the infrastructure” has been mentioned in the curriculum exam, a detailed discussion of this topic would go beyond the scope of this book and isn't directly related to Kubernetes. If you are more interested in the topic, have a look at the book [*Infrastructure as Code*](#) by Kief Morris (O'Reilly, 2025).

Understanding Extension Interfaces

[Chapter 2](#) laid the foundation for understanding the Kubernetes architecture, node types, and their cluster components. Kubernetes architecture

has been designed to be flexible, modular, and extensible so that the core functionality of the platform can be expanded. You can extend the cluster using *extensions*. Extensions are components that can be plugged into the platform to support custom functionality not native to Kubernetes. Those plug-ins usually include functions like network plug-ins, device plug-ins, and storage plug-ins.

This section talks about critical interfaces that you will need to understand as a Kubernetes administrator:

- [Container Network Interface \(CNI\)](#)
- [Container Runtime Interface \(CRI\)](#)
- [Container Storage Interface \(CSI\)](#)

Container Network Interface (CNI)

The CNI is a specification and the corresponding libraries for writing plug-ins to configure network interfaces in Linux containers. In Kubernetes, the CNI is responsible for establishing network connections between Pods running in the cluster. You will need to install a CNI plug-in in the control plane node(s) for them to function properly. [“Installing a Pod Network Add-on”](#) demonstrates the installation process of a CNI.

Popular choices include Calico, flannel, Cilium, and cloud-specific solutions like AWS VPC CNI and Azure CNI.

Container Runtime Interface (CRI)

The container runtime is responsible for managing the lifecycle of containers and ensuring that containers receive the resources they requested. Kubernetes uses containerd as its default container runtime, however, you can swap it out with a different container runtime if needed. The CRI is an API facilitating the interaction between Kubernetes and different container runtimes.

Common providers include containerd (the most popular), CRI-O, Docker Engine (via cri-dockerd), and Mirantis Container Runtime, with roughly 10–15 production-ready runtime options available that implement the CRI specification.

Container Storage Interface (CSI)

Despite the fact that Kubernetes provides a volume plug-in system, it was hard to integrate third-party storage solutions. In response, CSI was developed as a standard to implement plug-ins for integrating arbitrary block and file storage systems to containerized workloads.

Leading providers include cloud native options like AWS EBS CSI, Azure Disk CSI, and GCE Persistent Disk CSI, along with storage system drivers from NetApp, Pure Storage, Portworx, and Rook/Ceph, totaling over one hundred CSI drivers available in the ecosystem for various storage backends from cloud providers to traditional storage vendors.

Using kubeadm

The low-level command-line tool for performing cluster bootstrapping operations is called `kubeadm`. It is not meant for provisioning the underlying infrastructure.

To install `kubeadm`, follow the [installation instructions](#) in the official Kubernetes documentation. While not explicitly stated in the CKA frequently asked questions (FAQ) page, you can assume that the `kubeadm` executable has been preinstalled for you.

The following sections describe the processes for creating and managing a Kubernetes cluster on a high level and will use `kubeadm` heavily. For more detailed information, see the step-by-step Kubernetes reference documentation I will point out for each of the tasks.

Installing a Cluster

The most basic topology of a Kubernetes cluster consists of a single node that acts as the control plane and the worker node at the same time. By default, many developer-centric Kubernetes installations like minikube or Docker Desktop start with this configuration. While a single-node cluster may be a good option for a Kubernetes playground, it is not a good foundation for scalability and high-availability reasons. At the very least, you will want to create a cluster with a single control plane and one or many nodes handling the workload.

This section explains how to install a cluster with a single control plane and one worker node. You can repeat the worker node installation process to add more worker nodes to the cluster. You can find a full description of the [installation steps](#) in the official Kubernetes documentation. [Figure 4-1](#) illustrates the installation process.

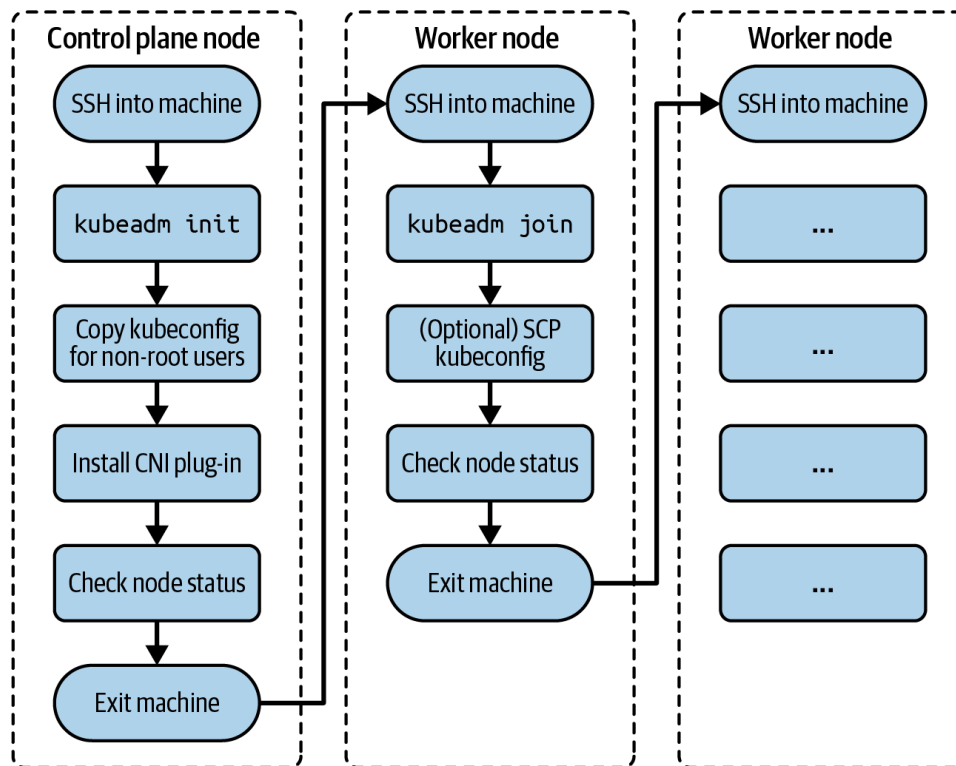


Figure 4-1. Process for a cluster installation

Initializing the Control Plane Node

Start by initializing the control plane on the control plane node. The control plane is the machine responsible for hosting the API server, etcd, and other components important to managing the Kubernetes cluster.

Open an interactive shell to the control plane node using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.10:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

Initialize the control plane using the `kubeadm init` command. You will need to add the following two command-line options: provide the IP addresses for the Pod network with the option `--pod-network-cidr`. With the option `--apiserver-advertise-address`, you can declare the IP address the API Server will advertise to listen on.

By default, `kubeadm` uses its own version to determine the control plane node's version. For example, if you use `kubeadm` version 1.31.1, then it will use version 1.31.1 for the initialized node. You can provide the desired Kubernetes version by providing the `--kubernetes-version` option, though it is recommended to use the `kubeadm` version you want to use for the nodes.

The console output renders a `kubeadm join` command. Keep that command around for later. It is important for joining worker nodes to the cluster in a later step.

RETRIEVING THE JOIN COMMAND FOR WORKER NODES

You can always retrieve the `join` command by running `kubeadm token create --print-join-command` on the control plane node should you lose it.

The following command uses `10.244.0.0/16` for the Classless Inter-Domain Routing (CIDR):

```
$ sudo kubeadm init --pod-network-cidr=10.244.0.0/16
```

```
...
```

To start using your cluster, you need to run the following as a regular user

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

You should now deploy a pod network to the cluster.

Run `"kubectl apply -f [podnetwork].yaml"` with one of the options listed at: <https://kubernetes.io/docs/concepts/cluster-administration/addons/>

Then you can join any number of worker nodes by running the following on \ each as root:

```
kubeadm join 172.16.0.5:6443 --token fi8io0.dtkzsy9kws56dmsp \
--discovery-token-ca-cert-hash \
sha256:cc89ea1f82d5ec460e21b69476e0c052d691d0c52cce83fbd7e403559c1ebdac
```

After the `init` command has finished, run the necessary commands from the console output:

```
$ mkdir -p $HOME/.kube
$ sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
$ sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Installing a Pod Network Add-on

You must deploy a [Container Network Interface \(CNI\) plug-in](#) so that Pods can communicate with each other. You can pick from a wide range of networking plug-ins listed in the [Kubernetes documentation](#). Popular plug-ins include flannel, Calico, and Cilium. Sometimes you will see the term *add-ons* in the documentation, which is synonymous with plug-in.

The exam will most likely ask you to install a specific add-on. Most of the installation instructions live on external web pages which are not permitted to be used during the exam. Make sure that you search for the relevant instructions in the official Kubernetes documentation. For example, you can find the [installation instructions for flannel on GitHub](#).

Installing flannel using kubectl

The following command installs the flannel objects via the released YAML manifest:

```
$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
```

The YAML manifest hard-codes the Pod CIDR to the value `10.244.0.0/16`. If you want to change that value, you'd have to download the YAML manifest first, edit the corresponding value, and then apply the manifest.

Installing flannel using Helm

You may decide to install flannel using Helm instead, either because you prefer the installation method or because you want to more conveniently provide a custom Pod CIDR. Helm offers the `--set` option to inject the custom value. The following command shows the use of Helm to install flannel. Refer to [“Working with Helm”](#) to learn more about the use of Helm:

```
$ kubectl create ns kube-flannel
$ kubectl label --overwrite ns kube-flannel pod-security.kubernetes.io/enforce=privileged
$ helm repo add flannel https://flannel-io.github.io/flannel/
$ helm install flannel --set podCidr="10.244.0.0/16" --namespace kube-flannel flannel/flannel
namespace/kube-flannel created
namespace/kube-flannel labeled
"flannel" has been added to your repositories
NAME: flannel
LAST DEPLOYED: Wed Jan 29 23:03:33 2025
NAMESPACE: kube-flannel
STATUS: deployed
```

Both installation methods, the YAML manifest and the Helm chart, create the flannel Pod into the namespace `kube-flannel`. Wait until the Pod observes the `Running` status. You can check on the Pod's details with the following command:

```
$ kubectl get pods -n kube-flannel
```

NAME	READY	STATUS	RESTARTS	AGE
kube-flannel-ds-h6455	1/1	Running	0	25s

Verify that the control plane node indicates the `Ready` status using the command `kubectl get nodes`. It might take a couple of seconds before the node transitions from the `NotReady` status to the `Ready` status. You have an issue with your node installation if the status transition does not occur. Refer to [Part VI](#) for debugging strategies:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	5m31s	v1.31.1

Exit the control plane node using the `exit` command:

```
$ exit
logout
...
```

Joining the Worker Nodes

Worker nodes are responsible for handling the workload scheduled by the control plane. Examples of workloads are Pods, Deployments, Jobs, and CronJobs. To add a worker node to the cluster so that it can be used, you will have to run a couple of commands, as described next.

Open an interactive shell to the worker node using the `ssh` command. The following command targets the worker node named `kube-worker-1` running Ubuntu 24.10:

```
$ ssh kube-worker-1
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

Run the `kubeadm join` command provided by the `kubeadm init` console output on the control plane node. The following command shows an example. Remember that the IP address, token, and SHA256 hash will be different for you:

```
$ sudo kubeadm join 172.16.0.5:6443 --token fi8io0.dtkzsy9kws56dmsp \
--discovery-token-ca-cert-hash \
sha256:cc89ea1f82d5ec460e21b69476e0c052d691d0c52cce83fbd7e403559c1ebdac
[preflight] Running pre-flight checks
[preflight] Reading configuration from the cluster...
[preflight] FYI: You can look at this config file with \
'kubectl -n kube-system get cm kubeadm-config -o yaml'
[kubelet-start] Writing kubelet configuration to file \
"/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with \
flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap...
```

This node has joined the cluster:

- * Certificate signing request was sent to apiserver and a response was received
- * The Kubelet was informed of the new secure connection details.

Run `'kubectl get nodes'` on the control plane to see this node join the cluster.

You won't be able to run the `kubectl get nodes` command from the worker node without copying the administrator kubeconfig file from the control plane node. Follow the [instructions](#) in the Kubernetes documentation to do so, or log back into the control plane node. Here, we are just going to log back into the control plane node. You should see that the worker node has joined the cluster and is in a `Ready` status:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	2m14s	v1.31.1
kube-worker-1	Ready	<none>	6m43s	v1.31.1

You can repeat the process for any other worker node you want to add to the cluster.

Managing a Highly Available Cluster

Single control plane clusters are easy to install; however, they present an issue when the node is lost. Once the control plane node becomes un-

available, any ReplicaSet running on a worker node cannot re-create a Pod due to the inability to talk back to the scheduler running on a control plane node. Moreover, clusters cannot be accessed externally anymore (e.g., via `kubectl`), as the API server cannot be reached.

High-availability (HA) clusters help with scalability and redundancy. For the exam, you will need a basic understanding about configuring them and their implications. Given the complexity of standing up an HA cluster, it's unlikely that you'll be asked to perform the steps during the exam. For a full discussion on setting up HA clusters, see the [Kubernetes documentation](#).

Stacked etcd Topology

The *stacked etcd topology* involves creating two or more control plane nodes where etcd is colocated on the node. [Figure 4-2](#) shows a representation of the topology with three control plane nodes.

Each of the control plane nodes hosts the API server, the scheduler, and the controller manager. Worker nodes communicate with the API server through a load balancer. I recommend that you operate this cluster topology with a minimum of three control plane nodes for redundancy reasons due to the tight coupling of etcd to the control plane node. By default, `kubeadm` will create an etcd instance when joining a control plane node to the cluster.

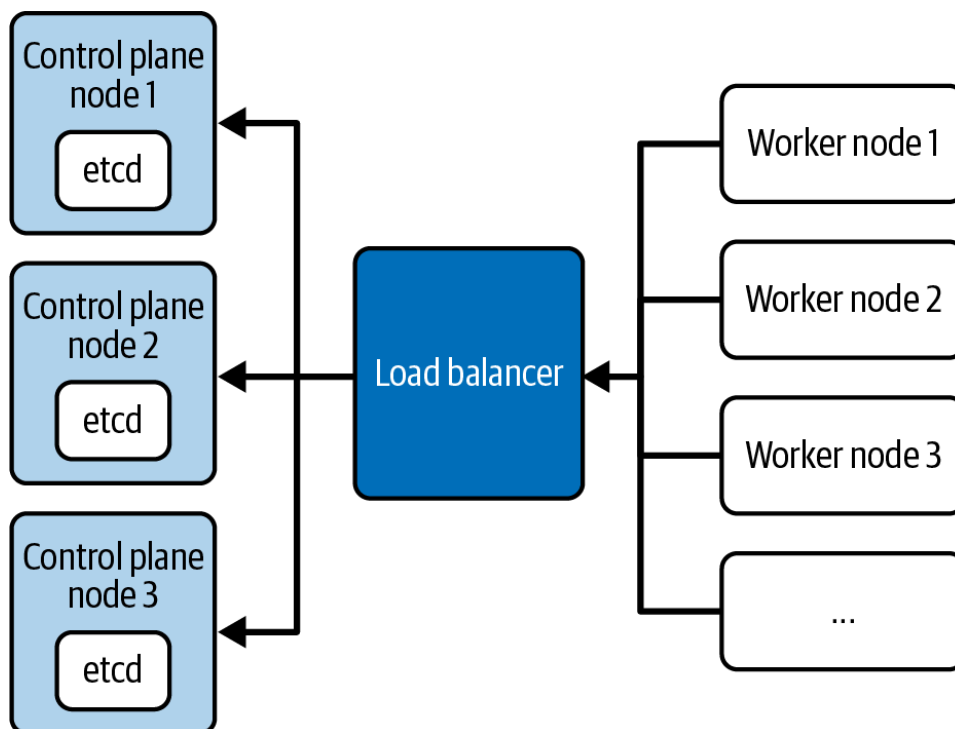


Figure 4-2. Stacked etcd topology with three control plane nodes

External etcd Node Topology

The *external etcd node topology* separates etcd from the control plane node by running it on a dedicated machine. [Figure 4-3](#) shows a setup with three control plane nodes, each of which runs etcd on a different machine.

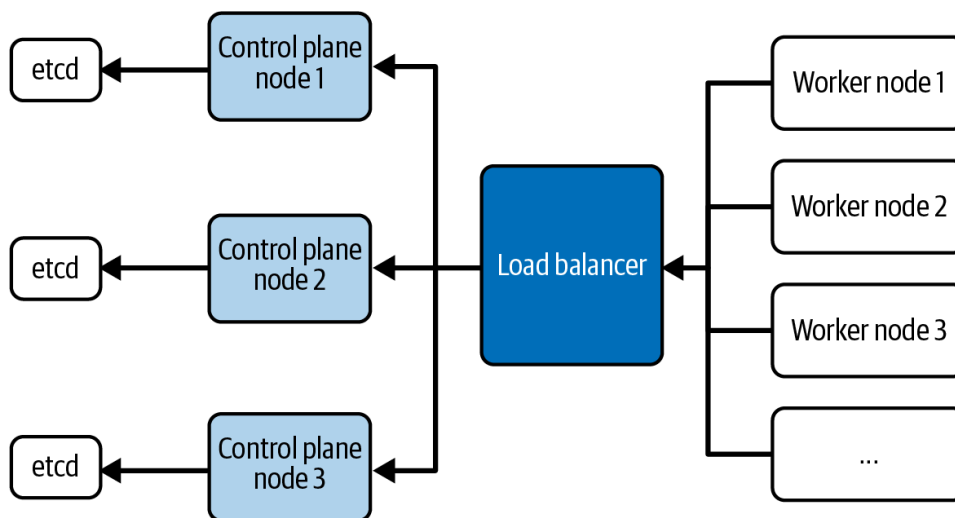


Figure 4-3. External etcd node topology

Similar to the stacked etcd topology, each control plane node hosts the API server, the scheduler, and the controller manager. The worker nodes communicate with them through a load balancer. The main difference here is that the etcd instances run on a separate host. This topology decouples etcd from other control plane functionality and therefore has less of an impact on redundancy when a control plane node is lost. As you can see in the illustration, this topology requires twice as many hosts as the stacked etcd topology.

Upgrading a Cluster Version

Over time, you will want to upgrade the Kubernetes version of an existing cluster to pick up bug fixes and new features. The upgrade process has to be performed in a controlled manner to avoid the disruption of workload currently in execution and to prevent the corruption of cluster nodes.

It is recommended to upgrade from a minor version to a next higher one (e.g., from 1.31.0 to 1.32.0), or from a patch version to a higher one (e.g., from 1.31.0 to 1.31.3). Abstain from jumping up multiple minor versions to avoid unexpected side effects. You can find a full description of the [upgrade steps](#) in the official Kubernetes documentation. [Figure 4-4](#) illustrates the upgrade process.

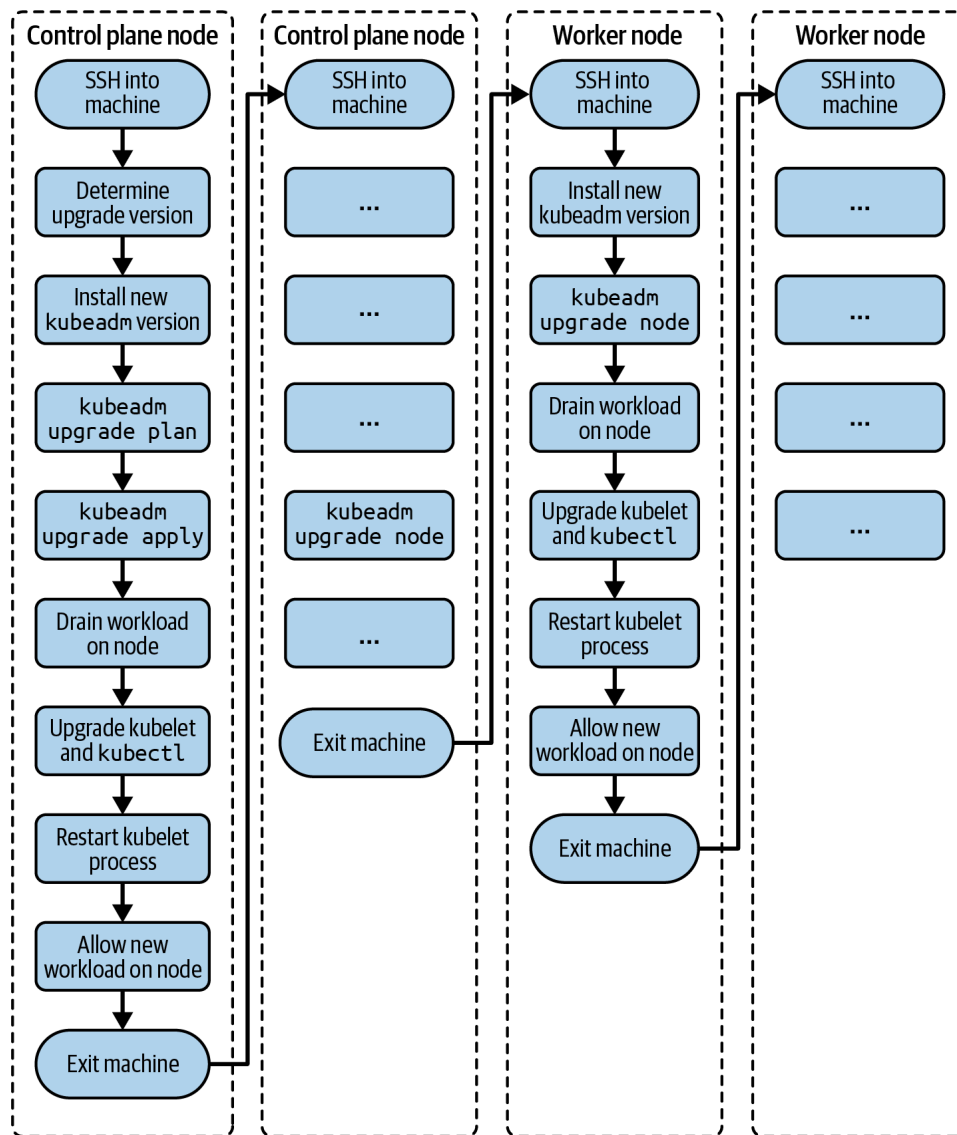


Figure 4-4. Process for a cluster version upgrade

Upgrading Control Plane Nodes

As explained earlier, a Kubernetes cluster may employ one or many control plane nodes to better support high-availability and scalability concerns. When upgrading a cluster version, this change needs to happen for control plane nodes one at a time.

Pick one of the control plane nodes that contains the kubeconfig file (located at `/etc/kubernetes/admin.conf`), and open an interactive shell to the control plane node using the `ssh` command. The following command targets the control plane node named `kube-control-plane` running Ubuntu 24.10:

```
$ ssh kube-control-plane
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)
...
```

First, check the nodes and their Kubernetes versions. In this setup, all nodes run on version 1.31.1. We are dealing with only a single control plane node and a single worker node:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	4m54s	v1.31.1
kube-worker-1	Ready	<none>	3m18s	v1.31.1

Start by upgrading the `kubeadm` version. Identify the version you'd like to upgrade to. On Ubuntu machines, you can use the following `apt-get` command. The version format usually includes a patch version (e.g., `1.20.7-00`). Check the Kubernetes documentation if your machine is running a different operating system:

```
$ sudo apt update
```

```
...
```

```
$ sudo apt-cache madison kubeadm
```

```
kubeadm | 1.31.5-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
kubeadm | 1.31.4-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
kubeadm | 1.31.3-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
kubeadm | 1.31.2-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
kubeadm | 1.31.1-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
kubeadm | 1.31.0-1.1 | https://pkgs.k8s.io/core:/stable:/v1.31/deb | Pack
```

Upgrade `kubeadm` to a target version. Say you'd want to upgrade to version `1.31.5-1.1`. The following series of commands installs `kubeadm` with that specific version and checks the currently installed version to verify:

```
$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get install
-y kubeadm=1.31.5-1.1 && sudo apt-mark hold kubeadm
```

```
Canceled hold on kubeadm.
```

```
...
```

```
Unpacking kubeadm (1.31.5-1.1) over (1.31.1-1.1) ...
```

```
Setting up kubeadm (1.31.5-1.1) ...
```

```
kubeadm set on hold.
```

```
$ sudo apt-get update && sudo apt-get install -y --allow-change-held-packag
kubeadm=1.31.5-1.1
```

```
...
```

```
kubeadm is already the newest version (1.31.5-1.1).
```

```
0 upgraded, 0 newly installed, 0 to remove and 94 not upgraded.
```

```
$ kubeadm version
```

```
kubeadm version: &version.Info{Major:"1", Minor:"31", GitVersion:"v1.31.5",
GitCommit:"af64d838aacd9173317b39cf273741816bd82377", GitTreeState:"clean",
BuildDate:"2025-01-15T14:39:21Z", GoVersion:"go1.22.10", Compiler:"gc", \
Platform:"linux/arm64"}
```

Check which versions are available to upgrade to and validate whether your current cluster is upgradable. You can see in the output of the following command that we could upgrade to version `1.31.5`:

```
$ sudo kubeadm upgrade plan
```

```
...
[upgrade] Fetching available versions to upgrade to
[upgrade/versions] Cluster version: 1.31.5
[upgrade/versions] kubeadm version: v1.31.5
I0130 22:26:53.887541 13574 version.go:261] remote version is \
much newer: v1.32.1; falling back to: stable-1.31
[upgrade/versions] Target version: v1.31.5
[upgrade/versions] Latest version in the v1.31 series: v1.31.
```

As described in the console output, we'll start the upgrade for the control plane. The process may take a couple of minutes. You may have to upgrade the CNI plug-in as well. Follow the provider instructions for more information:

```
$ sudo kubeadm upgrade apply v1.31.5
```

```
...
[upgrade/version] You have chosen to change the cluster version to "v1.31.5"
[upgrade/versions] Cluster version: v1.31.5
[upgrade/versions] kubeadm version: v1.31.5
...
[upgrade/successful] SUCCESS! Your cluster was upgraded to "v1.31.5". Enjoy

[upgrade/kubelet] Now that your control plane is upgraded, please proceed \
with upgrading your kubelets if you haven't already done so.
```

Drain the control plane node by evicting the workload. Any new workload won't be schedulable on the node until uncordoned:

```
$ kubectl drain kube-control-plane --ignore-daemonsets
```

```
node/kube-control-plane cordoned
WARNING: ignoring DaemonSet-managed Pods: kube-system/calico-node-qndb9, \
kube-system/kube-proxy-vpvm5
evicting pod kube-system/calico-kube-controllers-65f8bc95db-krp72
evicting pod kube-system/coredns-f9fd979d6-2brkq
pod/calico-kube-controllers-65f8bc95db-krp72 evicted
pod/coredns-f9fd979d6-2brkq evicted
node/kube-control-plane evicted
```

Upgrade the kubelet and the `kubectl` tool to the same version:

```
$ sudo apt-mark unhold kubelet kubectl && sudo apt-get update && sudo \
  apt-get install -y kubelet=1.31.5-1.1 kubectl=1.31.5-1.1 && sudo apt-mark
  hold kubelet kubectl
...
Setting up kubelet (1.31.5-1.1) ...
Setting up kubectl (1.31.5-1.1) ...
```

```
kubelet set on hold.  
kubectl set on hold.
```

Restart the kubelet process:

```
$ sudo systemctl daemon-reload  
$ sudo systemctl restart kubelet
```

Reenable the control plane node so that the new workload can become schedulable:

```
$ kubectl uncordon kube-control-plane  
node/kube-control-plane uncordoned
```

The control plane node should now show the usage of Kubernetes 1.31.5:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	21h	v1.31.5
kube-worker-1	Ready	<none>	21h	v1.31.1

Exit the control plane node using the `exit` command:

```
$ exit  
logout  
...
```

Upgrading Worker Nodes

Pick one of the worker nodes and open an interactive shell to the node using the `ssh` command. The following command targets the worker node named `kube-worker-1` running Ubuntu 24.10:

```
$ ssh kube-worker-1  
Welcome to Ubuntu 24.10 (GNU/Linux 6.11.0-8-generic aarch64)  
...
```

Upgrade `kubeadm` to a target version. This is the same command you used for the control plane node, as explained earlier:

```
$ sudo apt-mark unhold kubeadm && sudo apt-get update && sudo apt-get install  
-y kubeadm=1.31.5-1.1 && sudo apt-mark hold kubeadm  
Canceled hold on kubeadm.  
...
```

```

Unpacking kubeadm (1.31.5-1.1) over (1.31.1-1.1) ...
Setting up kubeadm (1.31.5-1.1) ...
kubeadm set on hold.
$ kubeadm version
kubeadm version: &version.Info{Major:"1", Minor:"31", GitVersion:"v1.31.5",
GitCommit:"af64d838aacd9173317b39cf273741816bd82377", GitTreeState:"clean",
BuildDate:"2025-01-15T14:39:21Z", GoVersion:"go1.22.10", Compiler:"gc", \
Platform:"linux/arm64"}

```

Upgrade the kubelet configuration:

```

$ sudo kubeadm upgrade node
[upgrade] Reading configuration from the cluster...
[upgrade] FYI: You can look at this config file with \
'kubectl -n kube-system get cm kubeadm-config -o yaml'
[preflight] Running pre-flight checks
[preflight] Skipping prepull. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[upgrade] Skipping phase. Not a control plane node.
[upgrade] Backing up kubelet config file to \
/etc/kubernetes/tmp/kubeadm-kubelet-config3058962439/config.yaml
[kubelet-start] Writing kubelet configuration to file \
"/var/lib/kubelet/config.yaml"
[upgrade] The configuration for this node was successfully updated!
[upgrade] Now you should go ahead and upgrade the kubelet package \
using your package manager.

```

Drain the worker node by evicting the workload. Any new workload won't be schedulable on the node until uncordoned:

```

$ kubectl drain kube-worker-1 --ignore-daemonsets
node/kube-worker-1 cordoned
WARNING: ignoring DaemonSet-managed Pods: kube-system/calico-node-2hrxg, \
kube-system/kube-proxy-qf6nl
evicting pod kube-system/calico-kube-controllers-65f8bc95db-kggbr
evicting pod kube-system/coredns-f9fd979d6-7zm4q
evicting pod kube-system/coredns-f9fd979d6-tlmhq
pod/calico-kube-controllers-65f8bc95db-kggbr evicted
pod/coredns-f9fd979d6-7zm4q evicted
pod/coredns-f9fd979d6-tlmhq evicted
node/kube-worker-1 evicted

```

Upgrade the kubelet and the `kubectl` tool with the same command used for the control plane node:

```

$ sudo apt-mark unhold kubelet kubectl && sudo apt-get update && sudo apt-g
install -y kubelet=1.31.5-1.1 kubectl=1.31.5-1.1 && sudo apt-mark hold ku
kubectl
...

```

```
Setting up kubelet (1.31.5-1.1) ...
Setting up kubect1 (1.31.5-1.1) ...
kubelet set on hold.
kubect1 set on hold.
```

Restart the kubelet process:

```
$ sudo systemctl daemon-reload
$ sudo systemctl restart kubelet
```

Reenable the worker node so that the new workload can become schedulable:

```
$ kubectl uncordon kube-worker-1
node/kube-worker-1 uncordoned
```

Listing the nodes should now show version 1.31.5 for the worker node. You won't be able to run `kubectl get nodes` from the worker node without copying the administrator kubeconfig file from the control plane node. Follow the [instructions](#) in the Kubernetes documentation to do so, or log back into the control plane node:

```
$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
kube-control-plane	Ready	control-plane	24h	v1.31.5
kube-worker-1	Ready	<none>	24h	v1.31.5

Exit the worker node using the `exit` command:

```
$ exit
logout
...
```

Summary

As a Kubernetes administrator, you need to be familiar with typical tasks involving the management of the cluster nodes. The primary tool for installing new nodes and upgrading a node version is `kubeadm`. The cluster topology of such a cluster can vary. For optimal results with redundancy and scalability, consider configuring the cluster with a high-availability setup that uses three or more control plane nodes and dedicated etcd hosts.

Exam Essentials

Understand why you need to provision infrastructure

Every cluster node needs to run on a physical or virtual machine. Provisioning the hardware is the job of an administrator, though you will not need to have hands-on experience for the exam. Explore the manual and automated approaches for provisioning hardware, as you will need it for installing a cluster.

Know how to create a Kubernetes cluster from scratch

Installing new cluster nodes and upgrading the version of an existing cluster node are typical tasks performed by a Kubernetes administrator. You do not need to memorize all the steps involved. The documentation provides a step-by-step, easy-to-follow manual for those operations. During the exam, pull up the relevant documentation and copy-paste the commands.

Practice the cluster upgrade process

The cluster upgrade process involves executing more commands than the installation process. It's important to remember that you only jump up by a single minor version or multiple patch versions before tackling the next higher version. I'd suggest you open the upgrade documentation page and walk through the process a couple of times.

Have a theoretical understanding of high-availability cluster topologies

High-availability clusters help with redundancy and scalability. For the exam, you will need to understand the different HA topologies, though it's unlikely that you'll have to configure one of them as the process would involve a suite of different hosts.

Sample Exercises

Solutions to these exercises are available in [Appendix A](#).

1. Create a cluster with four nodes: one control plane node, and three worker nodes.

Create a Pod named `nginx` that uses the container image

`nginx:1.27.4-alpine`.

Identify the node the Pod has been scheduled on.

Evict all Pods from the node that runs the Pod at once. Do not use the `kubectl delete pod` command to perform the operation. Ensure that the Pod is not running anymore.

Prerequisite: To create a cluster with four nodes using minikube, run the command `minikube start --nodes 4`.

2. Navigate to the directory *app-a/ch04/upgrade-cluster-version* of the checked-out GitHub repository [bmuschko/cka-study-guide](#).

Start up the virtual machines (VMs) running the cluster using the command `vagrant up`. The cluster consists of a single control plane node named `kube-control-plane`, and one worker node named `kube-worker-1`. Open an interactive shell into the control plane node and inspect the currently-used Kubernetes version by listing all the nodes. Upgrade all nodes of the cluster from Kubernetes 1.32.1 to 1.32.2. Once done, shut down the cluster using `vagrant destroy -f`.

Prerequisite: This exercise requires the installation of the tools [Vagrant](#) and a [VMware provider](#).